# ATLAS Traffic — Brief de proyecto

> Documento de especificación para construir el proyecto con Claude Code. Repo propio, desde cero. Entregable: pipeline de CV + dashboard comparativo multi-clip.

---

## 1. Qué es y qué lo diferencia

Un pipeline que procesa video de tránsito filmado desde un punto fijo, cuenta vehículos y peatones de forma única (no por frame), clasifica el nivel de congestión y consolida varios clips en un dataset comparable por día y horario.

**El diferencial no es detectar autos.** Eso lo hace cualquier tutorial. El diferencial son tres decisiones:

1. **Conteo único, direccional y por aproximación.** Línea virtual por cada brazo del cruce, con validación de sentido. Resuelve el problema real de los IDs que se rompen y permite reportar flujo desagregado por movimiento, no un total indiferenciado.
2. **Congestión definida como densidad + velocidad, no como volumen.** Mucho flujo a buena velocidad es tránsito fluido, no congestión. Mucha densidad con velocidad baja sí lo es. Esta distinción es lo que separa un demo de un análisis.
3. **Comparabilidad entre clips.** Cada corrida alimenta un dataset único con metadata de día y horario, de modo que la salida final responde "cómo varía el flujo de esta avenida" y no solo "detecté 47 autos".
4. **Atributos consultables por objeto.** Cada vehículo detectado queda registrado con tipo, color, sentido, momento de cruce y velocidad, más un recorte de imagen. Eso convierte el dashboard en una herramienta de consulta ("autos blancos que pasaron entre las 8 y las 8:30 en sentido norte") y no solo en un reporte agregado.

---

## 2. Decisiones cerradas

| Decisión | Definición |
|---|---|
| Punto de partida | Repo propio desde cero |
| Lenguaje | Python 3.11+ |
| Detección | Ultralytics YOLO ( `yolov8n` para iterar, `yolov8s` para la corrida final) |
| Tracking | ByteTrack |
| Inferencia | CPU |
| Config | Pydantic + YAML |
| Datos | Pandas → Parquet |
| Dashboard | Streamlit + Altair |
| Tests | pytest |
| Detector | abstraído detrás de interfaz (§4c) — no acoplar al código |
| Licencia | MIT en el código propio (ver nota AGPL en §8) |

**Nombre:** ATLAS Traffic. Repo `atlas-traffic`, paquete Python `atlas_traffic`.

**Escenario real.** Bocacalle de avenida en CABA (Recoleta), filmada desde balcón a ~6 pisos. No es un tramo recto: es un cruce con múltiples aproximaciones y flujos en distinta dirección. Esto no es un detalle — condiciona el diseño del conteo (§3b).

---

## 3. Modelo de datos

Esta es la parte que hay que cerrar antes de escribir código. Tres niveles, de crudo a agregado.

### 3.1 `configs/clips.yaml` — registro de clips

Se carga a mano al grabar. Es la fuente de verdad del contexto.

```
clips:
```

```yaml
  - id: 2026-09-15_0800_mar
    archivo: data/raw/2026-09-15_0800.mp4
    fecha: 2026-09-15
    hora_inicio: "08:00"
    dia_semana: martes
    tipo_dia: habil
    clima: despejado
    geometria: configs/geometria/encuadre_a.yaml
    notas: "hora pico mañana"
```

Campos: `id`, `archivo`, `fecha`, `hora_inicio`, `dia_semana`, `tipo_dia` (habil/finde), `clima`, `geometria`, `notas`.

`geometria` apunta al archivo que define líneas de conteo y ROI para ese encuadre. Clips filmados con la misma vista comparten el mismo archivo.

### 3.2 `outputs/runs/<clip_id>/tracks.parquet` — un registro por objeto

| Campo | Descripción |
|---|---|
| `track_id` | ID asignado por el tracker |
| `clase` | vehiculo / peaton (agrupando las clases COCO) |
| `clase_detalle` | car, truck, bus, motorcycle, person |
| `frame_inicio`, `frame_fin` | permanencia en cuadro |
| `cruzo_linea` | bool |
| `aproximacion` | id de la línea cruzada (ej. `av_norte`, `calle_este`) |
| `sentido` | entrante / saliente respecto del cruce |
| `ts_cruce` | segundo del video en que cruzó |
| `velocidad_px` | desplazamiento medio en px/frame |
| `color` | blanco / negro / gris / rojo / azul / verde / amarillo / indefinido |
| `color_confianza` | 0 a 1 — proporción de frames que votaron el color ganador |
| `thumb` | ruta al recorte representativo del objeto |

### 3.3 `outputs/runs/<clip_id>/intervalos.parquet` — un registro por ventana de 30s

| Campo | Descripción |
|---|---|
| `clip_id`, `bucket` | clave |
| `flujo_veh_min` | vehículos que cruzaron la línea, normalizado a por minuto |
| `flujo_peat_min` | ídem peatones |
| `densidad_media` | promedio de vehículos simultáneos dentro del ROI |
| `velocidad_media_px` | velocidad media de los tracks activos |
| `congestion` | LOW / MODERATE / HIGH |

### 3.4 `outputs/summary.parquet`

Unión de todos los `intervalos` con la metadata de `clips.yaml` pegada. Es lo único que consume el dashboard.

---

## 3b. Geometría del cruce y conteo multi-aproximación

Una sola línea de conteo no sirve acá. El cruce tiene varias aproximaciones y los flujos van en direcciones distintas; un total agregado esconde la información útil.

### Archivo de geometría

Un YAML por encuadre, en `configs/geometria/`:

```yaml
encuadre: encuadre_a
resolucion: [1920, 1080]
roi: [[x1,y1], [x2,y2], [x3,y3], [x4,y4]]   # polígono del área analizada
lineas:
```

```yaml
      - id: av_norte
        puntos: [[x1,y1], [x2,y2]]
        sentido_positivo: entrante        # qué significa cruzar de A hacia B
      - id: av_sur
        puntos: [[x1,y1], [x2,y2]]
        sentido_positivo: saliente
      - id: calle_este
        puntos: [[x1,y1], [x2,y2]]
        sentido_positivo: entrante
```

Las coordenadas se definen una vez por encuadre con un script auxiliar (`atlas-traffic geometry --clip <id>`) que abre un frame y permite marcar los puntos con el mouse. No hardcodear coordenadas en el código.

### Reglas de conteo

- Un track cuenta **una sola vez por línea**. Si el ID se rompe y reaparece, el nuevo track puede volver a contar — ese es el error residual que mide el chequeo manual de la Fase 2.
- El cruce se detecta por cambio de signo del producto cruzado entre el centroide del track y el segmento de línea, entre dos frames consecutivos.
- El sentido sale del signo de ese cambio. Un track que cruza y vuelve cuenta en ambos sentidos, que es lo correcto.
- Objetos cuyo centroide nunca entra al ROI se descartan. Eso elimina el tránsito del fondo de la avenida, que está demasiado lejos para medirse con fiabilidad.

### Qué habilita

El dashboard puede reportar flujo por movimiento: cuánto entra por la avenida desde el norte, cuánto sale hacia el este, qué proporción del total representa cada aproximación. Eso es análisis de tránsito real, no un contador.

---

## 4. Cómo se define la congestión

No usar umbrales absolutos inventados. El procedimiento:

**Paso 1 — Normalizar la velocidad, por encuadre.** La velocidad en píxeles depende del encuadre: un mismo auto a la misma velocidad real mide distinto si la cámara se movió. Se normaliza contra el percentil 90 de velocidad observado **dentro de cada encuadre**, que representa la condición de flujo libre de esa vista. Clips que comparten `geometria` se normalizan juntos.

Esto es un supuesto, y hay que documentarlo: asume que en todo encuadre existe al menos un momento de flujo libre. Es razonable si hay al menos un clip de baja densidad por encuadre. Si no lo hay, la normalización infla la velocidad relativa y el encuadre queda sesgado hacia LOW.

**Paso 2 — Calibrar umbrales con los propios datos.** Densidad y velocidad se cortan en los percentiles 33 y 66 del dataset completo, ya normalizado. Nada de números mágicos.

**Paso 3 — Clasificar con una matriz explícita:**

| Densidad | Velocidad | Nivel |
|---|---|---|
| Baja | cualquiera | LOW |
| Media | Alta | LOW |
| Media | Media / Baja | MODERATE |
| Alta | Alta | MODERATE |
| Alta | Media / Baja | HIGH |

Los umbrales calculados se persisten en `configs/calibracion.yaml` para que las corridas sean reproducibles y auditables. La matriz vive en el código, documentada, y es un parámetro discutible — no una verdad.

---

## 4b. Extracción de atributos (color)

El color es el único atributo que no viene gratis del detector. Procedimiento:

**Paso 1 — Muestreo.** Por cada track, tomar hasta 10 frames distribuidos a lo largo de su permanencia en cuadro. Descartar frames donde la caja esté parcialmente fuera del encuadre o solapada con otra caja por encima del 40%.

**Paso 2 — Región de interés.** No usar la caja completa: el borde captura fondo, sombra y asfalto. Recortar el 50% central de la caja. Para vehículos, sesgar la región hacia la mitad superior, que es carrocería y no sombra bajo el chasis.

**Paso 3 — Clasificación.** Convertir a HSV y clasificar contra una paleta de 8 valores. La regla de corte:

- Saturación baja → se decide por luminosidad: alta = blanco, media = gris, baja = negro
- Saturación media o alta → se decide por matiz: rojo, azul, verde, amarillo

**Paso 4 — Votación.** El color del track es la moda de los frames muestreados. `color_confianza` es la proporción de votos del ganador. Por debajo de 0.5, el color pasa a `indefinido`. Mejor abstenerse que inventar.

**Paso 5 — Evidencia.** Guardar el recorte del frame donde la caja fue más grande y menos ocluida, en `outputs/runs/<clip_id>/thumbs/<track_id>.jpg`. El dashboard los muestra en grilla al filtrar.

**Limitaciones a documentar explícitamente.** Blanco, plata y gris claro son indistinguibles bajo ciertas condiciones de luz; un vehículo blanco en sombra mide gris. La paleta es deliberadamente gruesa por eso. Los colores saturados (rojo, azul, amarillo) tienen precisión alta; la familia acromática es la que concentra el error. Vale la pena validar a mano una muestra de 30 tracks y reportar la precisión por color en el README.

**Lo que queda fuera de alcance.** Lectura de patentes: descartado por privacidad y porque a la distancia del balcón la resolución no da. Marca y modelo: requiere un clasificador entrenado aparte, no justifica el esfuerzo.

---

## 4c. Interfaz de detección — portabilidad de motor

**Por qué existe esta sección.** Ultralytics es AGPL-3.0. Para el MVP y el portfolio el uso es conforme: repo público, código abierto, demos locales. Si el proyecto deriva en producto comercial cerrado, hay que cambiar de motor o pagar licencia. Esa migración es barata si el detector está abstraído, y cara si está acoplado.

El costo de hacerlo bien hoy es de unas veinte líneas. El costo de arreglarlo después es medio pipeline.

### Contrato

```python
@dataclass(frozen=True)
class Deteccion:
    bbox: tuple[int, int, int, int]   # x1, y1, x2, y2
    clase: str                        # etiqueta normalizada del proyecto
    confianza: float
    track_id: int | None

class Detector(Protocol):
    def track(self, frame: np.ndarray) -> list[Deteccion]: ...
    def reset(self) -> None: ...      # limpia estado del tracker entre clips
```

### Reglas

- `UltralyticsDetector` es **una** implementación del protocolo, no el protocolo.
- El mapeo de clases COCO a las etiquetas del proyecto (`vehiculo`, `peaton`) vive en el adaptador, no en el resto del pipeline. Ningún módulo aguas abajo conoce el vocabulario del modelo.
- El detector se elige por nombre desde `configs/default.yaml` (`detector: ultralytics`), con un registro simple. Agregar un motor nuevo no debe requerir tocar `counting.py`, `metrics.py` ni `attributes.py`.
- Los tests se escriben contra el protocolo, con un `FakeDetector` que devuelve detecciones fijas. Eso permite testear conteo y métricas sin correr inferencia.

### Candidatos de reemplazo

Si en algún momento hace falta migrar, los motores con licencia permisiva (Apache 2.0) son RF-DETR, RT-DETR y YOLOX. Rendimiento comparable, sin las obligaciones de la AGPL.

---

## 4d. Regla de alerta — circulación en sentido contrario

Una sola regla, deliberadamente. No es un motor de reglas ni una infraestructura de alertas: es una función sobre datos que el pipeline ya produce. Su propósito es demostrar que el sistema detecta **eventos**, no solo cuenta objetos.

### Lógica

Cada línea en el archivo de geometría declara su `sentido_positivo`. Un track que cruza una línea en el sentido opuesto al declarado genera un evento. Para evitar falsos positivos:

- Ignorar tracks con menos de N frames de historia (ruido del tracker)

- Ignorar tracks cuyo desplazamiento total sea menor a un umbral en píxeles (vehículo detenido con jitter)
- Exigir que el cruce sea limpio: el centroide debe estar a un lado de la línea durante al menos 3 frames antes y después

### Salida

`outputs/runs/<clip_id>/eventos.parquet`

| Campo | Descripción |
|---|---|
| `clip_id`, `track_id` | clave |
| `ts` | segundo del video |
| `tipo` | `sentido_contrario` |
| `aproximacion` | línea involucrada |
| `thumb` | recorte de evidencia |

El dashboard muestra los eventos en una tabla con el recorte al lado. Cada evento es verificable a ojo.

### Disciplina de alcance

No agregar más reglas en el MVP. Permanencia anómala, aglomeración e intrusión en zona son extensiones obvias, pero cada una suma superficie de error y tiempo de calibración. Una regla que funciona bien demuestra más que cuatro que fallan.

---

## 5. Estructura del repo

```
atlas-traffic/
├── src/atlas_traffic/
│   ├── cli.py            # entrypoint
│   ├── config.py         # modelos Pydantic + carga de YAML
│   ├── clips.py          # lectura y validación de clips.yaml
│   ├── geometry.py       # ROI, líneas de conteo, editor interactivo de puntos
│   ├── detect.py         # protocolo Detector + adaptador Ultralytics
│   ├── counting.py       # línea virtual, cruce, sentido, conteo único
│   ├── attributes.py     # extracción de color + recortes de evidencia
│   ├── alerts.py         # regla de sentido contrario → eventos.parquet
│   ├── metrics.py        # densidad, velocidad, agregación por intervalo
│   ├── congestion.py     # calibración de umbrales + clasificación
│   ├── aggregate.py      # consolidación multi-clip → summary.parquet
│   └── render.py         # video anotado + overlay de métricas
├── dashboard/app.py
├── configs/
│   ├── default.yaml
│   ├── clips.yaml
│   ├── geometria/        # un YAML por encuadre
│   └── calibracion.yaml  # generado
├── data/raw/             # gitignored
├── outputs/              # gitignored salvo muestras
├── tests/
├── docs/assets/
├── pyproject.toml
├── Dockerfile
└── README.md
```

El CLI expone: `run`, `batch`, `geometry`, `calibrate`, `dashboard`.

---

## 6. Plan de fases

Cada fase tiene criterio de aceptación. No pasar a la siguiente sin cumplirlo.

**Fase 0 — Scaffolding.** Estructura, `pyproject.toml`, CLI vacío con subcomandos, config Pydantic, pytest corriendo. → *Acepta cuando:* `python -m atlas_traffic --help` lista los subcomandos y `pytest` pasa en verde.

**Fase 0b — Protocolo de detección.** Definir `Detector`, `Deteccion` y `FakeDetector` según §4c, antes de integrar ningún modelo real. → *Acepta cuando:* existe un test que ejercita el protocolo con `FakeDetector` sin importar Ultralytics en ningún lado.

**Fase 1 — Detección y tracking.** `UltralyticsDetector` implementando el protocolo, con YOLO + ByteTrack sobre un clip de 20s. Salida cruda a `tracks.parquet`. → *Acepta cuando:* el parquet tiene IDs estables y clases correctas sobre el clip corto.

**Fase 1b — Geometría.** Comando `geometry` que abre un frame del clip y permite marcar ROI y líneas de conteo con el mouse,

guardando el YAML de encuadre. → *Acepta cuando:* existe `configs/geometria/encuadre_a.yaml` con ROI y al menos tres líneas, y el render de debug las dibuja sobre el frame en la posición correcta.

**Fase 2 — Conteo multi-aproximación.** Detección de cruce por línea, sentido, conteo único, filtro por ROI. → *Acepta cuando:* el conteo automático por aproximación coincide con el conteo manual del clip de 20s, con margen de ±10% en cada línea. Este chequeo es obligatorio y hay que hacerlo a mano una vez, línea por línea.

**Fase 3 — Métricas por intervalo.** Densidad, velocidad, agregación en ventanas de 30s. → *Acepta cuando:* `intervalos.parquet` se genera y los números son plausibles al mirarlos.

**Fase 3b — Atributos.** Extracción de color por votación y guardado de recortes, según §4b. → *Acepta cuando:* sobre una muestra manual de 30 tracks, los colores saturados aciertan por encima del 85% y la familia acromática por encima del 65%. Registrar esos números, van al README.

**Fase 3c — Regla de alerta.** Detección de circulación en sentido contrario según §4d, con los tres filtros antifalso-positivo. → *Acepta cuando:* sobre los clips disponibles, cada evento disparado es verificable mirando el recorte. Cero falsos positivos en el set de prueba, aunque haya que endurecer umbrales para lograrlo. Es preferible perder un evento real a mostrar uno falso en un demo.

**Fase 4 — Congestión.** Normalización de velocidad por encuadre, calibración por percentiles, matriz de clasificación, persistencia de umbrales. → *Acepta cuando:* el clip de hora pico da HIGH y el de domingo temprano da LOW, sin tocar los umbrales a mano.

**Fase 5 — Render.** Video anotado con cajas, IDs, líneas de conteo, ROI y contador por aproximación en pantalla. → *Acepta cuando:* el video sale y el GIF de muestra es presentable.

**Fase 6 — Batch multi-clip.** `batch` procesa todos los clips de `clips.yaml` y consolida en `summary.parquet`. → *Acepta cuando:* un solo comando reprocesa todo el dataset de punta a punta.

**Fase 7 — Dashboard.** Streamlit, dos pestañas.

*Pestaña A — Panorama,* sobre `summary.parquet`: serie temporal por clip, comparativa hábil vs finde, distribución de niveles de congestión por franja horaria, tabla de clips.

*Pestaña B — Eventos:* tabla de `eventos.parquet` con recorte de evidencia al lado de cada registro, filtrable por clip y aproximación.

*Pestaña C — Consulta,* sobre la unión de todos los `tracks.parquet`. Filtros en la barra lateral: clip o rango de fechas, ventana horaria dentro del video, tipo de vehículo, color, sentido, confianza mínima de color. Salida: conteo del subconjunto, grilla de recortes como evidencia visual, y botón de descarga a CSV.

→ *Acepta cuando:* podés responder "cuántos autos blancos pasaron en sentido norte entre el minuto 2 y el 5" en tres clics, y verificar el resultado mirando los recortes.

**Fase 8 — Cierre.** Tests de las funciones puras (conteo, clasificación, agregación), CI en GitHub Actions, Dockerfile, README con capturas, GIF y sección de limitaciones. → *Acepta cuando:* el repo se ve terminado desde afuera.

---

## 7. Prompt inicial para Claude Code

> Voy a construir ATLAS Traffic, un pipeline de visión por computadora para analizar tránsito en una bocacalle de avenida. Te paso el brief completo en `brief-traffic-flow.md` — leelo entero antes de escribir nada.
>
> Arrancamos por la Fase 0: scaffolding. Quiero la estructura de carpetas, `pyproject.toml` con las dependencias, el CLI con los subcomandos `run`, `batch`, `geometry`, `calibrate` y `dashboard` (por ahora vacíos), los modelos Pydantic de configuración y pytest funcionando.
>
> No implementes lógica de detección todavía. Cuando termines, corré los tests y mostrame el árbol de archivos.

Después de cada fase, el patrón es el mismo: nombrar la fase, pedir el criterio de aceptación explícito, y no avanzar hasta verlo cumplido.

---

## 8. Notas que no son código

**AGPL de Ultralytics — postura del MVP.** El código propio va MIT. Ultralytics es AGPL-3.0, y para este MVP el uso es plenamente conforme: repo público y abierto, demos locales, sin distribución de producto cerrado ni servicio pago. No hay zona gris acá.

La obligación aparece recién si el proyecto se convierte en producto comercial cerrado o en servicio alojado. Para ese escenario, el protocolo `Detector` de §4c es el seguro: permite cambiar a RF-DETR, RT-DETR o YOLOX (Apache 2.0) sin reescribir el pipeline. La debida diligencia legal se hace cuando haya un cliente real, no antes.

**Qué queda fuera de alcance, a propósito.** Reconocimiento facial, lectura de patentes asociada a personas y seguimiento de individuos entre cámaras. No por capacidad técnica, sino porque mueven el proyecto a una categoría regulatoria distinta: la Ley 25.326 sigue vigente en Argentina y hay varios proyectos de reforma en el Congreso que endurecen el régimen, ninguno aprobado todavía. El conteo agregado y la detección de eventos de tránsito no tratan datos personales identificables, y ese es el terreno donde este sistema se mueve con comodidad.

**Privacidad.** Antes de subir el video o el GIF de demo a un repo público, revisar que no se distingan patentes ni caras. Bajar resolución en la muestra, o elegir un tramo donde no haya nada identificable. Vale la pena mencionar en el README que se tuvo en cuenta — suma.

**Sección de limitaciones en el README.** El clip nocturno va acá, no en el demo principal. Documentar que la detección degrada con poca luz, con la métrica comparada, muestra criterio. Esconderlo no.

**Consistencia de encuadre.** Lo ideal es que todos los clips compartan la misma vista: marcar la posición del teléfono en la baranda con cinta. Si hay encuadres distintos, cada uno necesita su propio archivo de geometría y su propia normalización de velocidad, y la comparación entre encuadres queda sujeta al supuesto del §4. Menos encuadres, menos supuestos.

**Filmar sin vidrio de por medio.** El reflejo de la ventana le mete ruido a la clasificación de color. Abrir la ventana o apoyar el lente contra el vidrio.

**Orden de grabación.** Grabar primero el clip de hora pico y desarrollar todo con ese. Recién con el conteo funcionando salir a buscar los demás. Si el ángulo del balcón no sirve, mejor descubrirlo con un video filmado que con seis.